Белорусский государственный университет

Факультет прикладной математики и информатики

Лабораторная работа №2

Основы обработки цифровых изображений

Вариант 11: Поэлементные операции и локальная пороговая обработка

Выполнил:

студент 3 курса

группы 4а

Молокович Александр

Минск, 2026

Содержание

1 Введение **2**

2 Математические основы и влияние на изображение **2**

2.1 Поэлементные операции 2

2.2 Локальная пороговая обработка (Адаптивная бинаризация) 3

3 Стек технологий **4**

4 Архитектура и оптимизация вычислений **4**

4.1 Интегральное изображение (для Adaptive Mean) 5

4.2 Сепарабельные ядра (для Adaptive Gaussian) 5

5 Возникшие проблемы и их решения **5**

6 Выводы **6**

1. Введение

Цель работы: изучить алгоритмы пространственной обработки цифровых изображений, реализовать методы поэлементного преобразования яркости и локальной пороговой бинаризации с применением математических оптимизаций.

Задачи:

1. Математически описать и реализовать линейное контрастирование и гамма-коррекцию.

2. Реализовать алгоритмы локальной бинаризации: адаптивное среднее и адаптивный фильтр Гаусса.

3. Применить алгоритмические оптимизации (интегральное изображение, сепарабельные ядра) для снижения асимптотической сложности локальных фильтров.

4. Разработать кроссплатформенное графическое приложение для демонстрации работы алгоритмов на тестовых изображениях.

2. Математические основы и влияние на изображение

В рамках работы алгоритмы реализованы на уровне базовых матричных операций. Перед выполнением пороговой обработки цветные изображения переводятся в градации серого по стандарту Luminance (яркости):

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \tag{1}$$

2.1. Поэлементные операции

Поэлементные операции модифицируют каждый пиксель независимо от его соседей.

1. Линейное контрастирование (Растяжение гистограммы)
Если изображение снято в условиях плохой видимости (например, туман), его гистограмма «сжата» в узком диапазоне яркостей. Метод находит ми-

нимальное (f_{min}) и максимальное (f_{max}) значения яркости и линейно отображает их на полный диапазон $[0, 255]$:

$$f'(m,n) = \frac{f(m,n) - f_{min}}{f_{max} - f_{min}} \cdot 255 \tag{2}$$

Визуальный эффект: Блёклые, серые изображения приобретают глубокие тени и яркие света, контуры объектов становятся чётко различимыми.

2. Гамма-коррекция (Степенное преобразование)

Нелинейная операция, компенсирующая особенности человеческого зрения. Значения пикселей нормализуются к диапазону $[0, 1]$, возводятся в степень $1/\gamma$, и масштабируются обратно:

$$f'(m,n) = \left(\frac{f(m,n)}{255}\right)^{\frac{1}{\gamma}} \cdot 255 \tag{3}$$

Визуальный эффект: При $\gamma > 1$ изображение осветляется, «вытягивая» детали из глубоких теней (функция выпуклая вверх). При $\gamma < 1$ изображение затемняется, спасая пересвеченные участки.

2.2. Локальная пороговая обработка (Адаптивная бинаризация)

Глобальный порог (одно значение для всего фото) не работает на изображениях с неравномерным освещением (например, текст, на который падает тень). Локальная бинаризация рассчитывает индивидуальный порог $T(x, y)$ для каждого пикселя на основе окрестности (блока размером $W \times W$). Если пиксель темнее порога, он становится чёрным (0), иначе - белым (255).

Для исключения появления цифрового шума на однородном фоне из расчётного порога вычитается константа-предохранитель C.

1. Метод адаптивного среднего (Adaptive Mean)

Порогом выступает среднее арифметическое всех пикселей в блоке:

$$T(x,y) = \frac{1}{W^2} \sum_{i=-\frac{W}{2}}^{\frac{W}{2}} \sum_{j=-\frac{W}{2}}^{\frac{W}{2}} f(x+i, y+j) - C \tag{4}$$

Визуальный эффект: Текст в тени становится читаемым, однако контуры

букв могут получаться «угловатыми» из-за равного веса всех пикселей в блоке.

2. Метод адаптивного Гаусса (Adaptive Gaussian)
Порог рассчитывается как взвешенная сумма пикселей блока. Веса распределены по закону нормального распределения: центральные пиксели имеют максимальный вес, периферийные - минимальный. *Визуальный эффект:* Границы объектов (букв) получаются плавными, а влияние случайного тёмного шума на краях блока нивелируется. Гаусс даёт наиболее чистый результат бинаризации.

3. Стек технологий

Для реализации приложения отказано от использования готовых алгоритмов компьютерного зрения (библиотеки `OpenCV`). Все вычисления производятся «с нуля». Использовались следующие технологии:

- **Python 3.10+** — основной язык программирования.
- **NumPy** — библиотека для высокопроизводительных векторных и матричных вычислений. Служит математическим ядром логики.
- **Streamlit** — современный фреймворк для быстрого создания реактивных веб-интерфейсов. Обеспечивает кроссплатформенность и работу в браузере.
- **Pillow (PIL)** — для декодирования загружаемых изображений в массивы пикселей.

4. Архитектура и оптимизация вычислений

Прямолинейная реализация локальных фильтров требует $O(W^2)$ операций умножения и сложения для каждого пикселя изображения. Для изображения 1000x1000 и блока 51x51 это приводит к миллиардам итераций, что вызывает зависание программы. Для решения проблемы были применены математические оптимизации.

4.1. Интегральное изображение (для Adaptive Mean)

Для расчёта суммы в блоке любого размера за $O(1)$ операций предварительно строится интегральная матрица I_Σ, где каждый элемент (x, y) равен сумме всех пикселей левее и выше него. Тогда сумма произвольного прямоугольного блока рассчитывается всего за 4 операции:

$$Sum = I_\Sigma(x_2, y_2) - I_\Sigma(x_2, y_1 - 1) - I_\Sigma(x_1 - 1, y_2) + I_\Sigma(x_1 - 1, y_1 - 1) \quad (5)$$

4.2. Сепарабельные ядра (для Adaptive Gaussian)

Двумерное ядро Гаусса K_{2D} обладает математическим свойством сепарабельности (разделимости), то есть может быть представлено как векторное произведение вектора-столбца на вектор-строку: $K_{2D} = K_y \cdot K_x^T$.

Это позволяет заменить сложную 2D-свёртку на два последовательных 1D-прохода:

```
filtered_h = np.sum(windows_h * kernel_1d, axis=2)
local_mean = np.sum(windows_v * kernel_1d, axis=2)
```

Данная архитектурная особенность снижает вычислительную сложность алгоритма с $O(W^2)$ до $O(2W)$. Например, для блока 21x21 количество операций на пиксель падает с 441 до 42, ускоряя код на порядок.

5. Возникшие проблемы и их решения

- **Ошибки переполнения типов данных.** При математических операциях значения пикселей типа `uint8` (0-255) часто переполнялись (например, $250 + 10 = 4$). *Решение:* Перед вычислениями матрицы принудительно кастуются к типу `float32`. По завершении преобразований применяется функция `np.clip(..., 0, 255)` и возврат к типу `uint8`.

- **Деление на ноль в линейном контрастировании.** Если изображение абсолютно однородно (например, чёрный квадрат), разница $f_{max} - f_{min}$ равна нулю. *Решение:* Использование маскирования `NumPy`: элементы разницы, равные 0, программно заменяются на 1.

- **Шум при бинаризации на однотонном фоне.** Без корректировки даже микроскопический шум матрицы камеры заставлял фильтр находить «ложные контуры» на чистом листе бумаги. *Решение:* Внедрение пользовательского параметра C (константы сдвига), обеспечивающего запас толерантности алгоритма к фоновому шуму.

6. Выводы

В ходе выполнения лабораторной работы были глубоко изучены и реализованы алгоритмы пространственной фильтрации и цветокоррекции изображений.

Доказано на практике, что локальная адаптивная бинаризация превосходит глобальные методы при работе с реальными фотографиями, содержащими неравномерное освещение и тени. Визуальный анализ показал, что взвешенное усреднение по Гауссу формирует более гладкие и чёткие контуры текста по сравнению с простым арифметическим усреднением.

Ключевым достижением работы является реализация алгоритмов с применением интегральных изображений и сепарабельных ядер. Это позволило добиться высокой производительности ($O(1)$ для Mean и $O(W)$ для Гаусса) исключительно средствами базовой математики (NumPy), без привлечения оптимизированных библиотек машинного зрения (OpenCV).